

Министерство образования и науки Российской Федерации  
Санкт-Петербургский политехнический университет Петра Великого  
Институт компьютерных наук и кибербезопасности  
Высшая школа технологий искусственного интеллекта  
Направление: 02.03.01 Математика и компьютерные науки

Отчет о выполнении лабораторной работы №2  
по дисциплине «Математическая логика и теория  
формальных языков»  
«Регулярные выражения»  
Вариант №8

Выполнила студентка группы  
№5130201/30101  
Проверил

Лаишевкина А. М. \_\_\_\_\_  
Востров А. В. \_\_\_\_\_

«\_\_\_\_\_» \_\_\_\_\_ 2026г.

Санкт-Петербург, 2026

# Содержание

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Введение</b>                                              | <b>3</b>  |
| <b>1 Математическое описание</b>                             | <b>4</b>  |
| 1.1 Регулярные выражения и регулярные множества . . . . .    | 4         |
| 1.2 Регулярное выражение для титульного листа . . . . .      | 5         |
| 1.3 Конечные автоматы . . . . .                              | 6         |
| 1.3.1 Недетерминированные конечные автоматы . . . . .        | 6         |
| 1.3.2 Детерминированные конечные автоматы . . . . .          | 10        |
| <b>2 Особенности реализации</b>                              | <b>13</b> |
| 2.1 Структуры данных . . . . .                               | 13        |
| 2.2 Инициализация автомата . . . . .                         | 13        |
| 2.3 Функции . . . . .                                        | 22        |
| 2.3.1 Вспомогательные функции . . . . .                      | 22        |
| 2.3.2 Функции, проверяющие совпадение символов . . . . .     | 23        |
| 2.3.3 Функция, реализующая логику конечного автомата . . . . | 23        |
| 2.3.4 Функции для генерации строк . . . . .                  | 24        |
| 2.3.5 Меню . . . . .                                         | 26        |
| <b>3 Результаты работы программы</b>                         | <b>28</b> |
| <b>Заключение</b>                                            | <b>30</b> |
| <b>Список литературы</b>                                     | <b>31</b> |

# Введение

**Цель:** реализовать программу, которая проверяет введенный текст на соответствие регулярному выражению через реализацию конечного автомата.

**Вариант 8:** Проверка корректности титульного листа лабораторной работы, курсовой работы или практики.

## **Задачи:**

1. Построить регулярное выражение, соответствующее варианту.
2. Построить недетерминированный конечный автомат, реализующий распознавание этого регулярного выражения.
3. Детерминировать полученный конечный автомат.
4. Реализовать программу, которая проверяет введенный текст через реализацию конечного автомата.
5. Реализовать функцию случайной генерации верной строки по полученному конечному автомату

# 1 Математическое описание

## 1.1 Регулярные выражения и регулярные множества

**Регулярные выражения** – это формальный язык шаблонов для поиска и выполнения манипуляций с подстроками в тексте; формула, задающая правило поиска подстрок в потоке символов. Они строятся из следующих элементов:

- **Базовые компоненты:**

- $\emptyset$  – пустое множество;
- $\varepsilon$  – пустая цепочка;
- $a \in \Sigma$  – любой символ из словаря

- **Операции:**

- *Объединение* ( $R_1 + R_2$  или  $R_1 | R_2$ ) – множество цепочек, принадлежащих  $R_1$  или  $R_2$ .
- *Конкатенация* ( $R_1 R_2$ ) – множество цепочек, образованных соединением цепочки из  $R_1$  и цепочки из  $R_2$ .
- *Итерация Клини* ( $R^*$ ) – множество цепочек, состоящих из нуля или более повторений цепочек из  $R$ .
- *Усеченная итерация* ( $R^+$ ) – множество цепочек, состоящих из одного или более повторений цепочек из  $R$ .

**Приоритет операций:**

1. Итерация ( $*$ ) и усеченная итерация ( $^+$ );
2. Конкатенация;
3. Объединение ( $+$  или  $|$ ).

Скобки изменяют порядок выполнения операций.

**Регулярные множества** – это бесконечные множества цепочек, построенных из символов конечного словаря по определенным правилам. Регулярные множества также называют регулярными языками, так как они являются множествами цепочек, построенных над конечным словарем.

**Примеры:**

- регулярное выражение  $ab + ba^*$  задает множество:

$$ab \cup b, ba, baa, baaa, \dots$$

- Выражение  $(ac)^*b + c^*$  описывает объединение двух множеств:

- Цепочки вида  $(ac)^nb$  (например,  $b, acb, acacb, \dots$ );
- Цепочки из любого числа символов  $c$  (включая  $\varepsilon$ ).

**Теорема Клини.** Классы регулярных множеств и автоматных языков совпадают.

**Следствия:**

- Любой язык, распознаваемый КА, может быть задан регулярным выражением.
- Для любого регулярного языка существует задающий его КА.

## 1.2 Регулярное выражение для титульного листа

Для титульного листа используется следующее регулярное выражение:

```

1 Министерство образования и науки Российской Федерации\n
2 Санкт-Петербургский политехнический университет Петра Великого\n
3 (([А-ЯЁ][а-яё ]+ институт)|(Институт [а-яё ]+))\n
4 Высшая школа [а-яё ]+\n
5 Направление: [0-9][0-9].[0-9][0-9].[0-9][0-9] [А-ЯЁ][а-яё ]+\n
6 Отчет о выполнении (лабораторной работы №[1-9][0-9]|курсовой работы|практики
  )\n
7 <<[А-ЯЁ][а-яё ]+>>\n
8 (Вариант №[1-9][0-9]\n)?
9 Выполнил(а)? студент(ка)? группы №
  [0-9][0-9][0-9][0-9][0-9][0-9][0-9]/[0-9][0-9][0-9][0-9][0-9] [А-ЯЁ][а-яё
  ]+ [А-ЯЁ]. [А-ЯЁ].\n
10 Проверил(а)? [А-ЯЁ][а-яё]+ [А-ЯЁ]. [А-ЯЁ].\n
11 <<[_]+>> [_]+ [0-9][0-9][0-9][0-9]г.\n
12 Санкт-Петербург, [0-9][0-9][0-9][0-9]
```

[А-ЯЁ] – одна русская заглавная буква.

[а-яё ] – одна русская строчная буква или пробел.

[а-яё ]+ – последовательность из одной или более русских строчных букв и пробелов.

[0-9] – одна цифра (от 0 до 9).

[1-9] – одна ненулевая цифра (от 1 до 9).

[\_]+ – последовательность из одного или более символов подчеркивания.

(а)? – символ «а» может присутствовать или отсутствовать (опционально).

(ка)? – символы «ка» могут присутствовать или отсутствовать.

| – оператор <<ИЛИ>>.

\n – символ перевода строки.

<<, >>, №, /, г., . – фиксированные литералы.

## 1.3 Конечные автоматы

**Конечный автомат** – это формальная модель преобразователей информации с конечной памятью. Выход конечного автомата зависит не только от входа, но и от цепочки входных сигналов, которые были поданы раньше, с момента начала функционирования автомата.

Формально конечный автомат можно определить следующим образом:

$$A = (S, \Sigma, \sigma, s_0, F), \text{ где}$$

$S$  – конечное множество состояний,  
 $\Sigma$  – конечное множество входных сигналов,  
 $\sigma : S \times \Sigma \rightarrow S$  – функция переходов,  
 $s_0$  – начальное состояние ( $s_0 \in S$ ),  
 $F$  – множество допускающих значений.

Автоматы делятся на детерминированные и недетерминированные. У детерминированного автомата в отличие от недетерминированного любой переход однозначно определяется по текущему состоянию и входному сигналу.

### 1.3.1 Недетерминированные конечные автоматы

Недетерминированный конечный автомат (НДКА) определяется как:

$$(S_1, \Sigma, \sigma, s_{0_1}, F), \text{ где:}$$

$S_1$  – конечное множество состояний,  
 $\Sigma$  – конечный входной алфавит,  
 $\sigma : S_1 \times (\Sigma \cup \varepsilon) \rightarrow 2^{S_1}$  – функция переходов,  
 $s_{0_1} \in S_1$  – начальное состояние,  
 $F \subseteq S_1$  – множество допускающих состояний.

В данной работе:

- $S_1 = \{\text{start, minister, n\_university, university, n\_institut, institut\_1\_1, institut\_1\_2, institut\_1\_3, institut\_2\_1, institut\_2\_2, institut\_2\_3, n\_school, school\_1, school\_2, school\_3, n\_program, program\_1, program\_2, program\_3, program\_4, n\_report, report\_1, report\_2, report\_3, report\_4, n\_name, name\_1, name\_2, name\_3, n\_variant, variant\_1, variant\_2, variant\_3, n\_student, student\_1, student\_1.5, student\_2, student\_3, student\_3.5, student\_4, student\_5, student\_6, student\_7, student\_8, student\_9, student\_10, student\_11, student\_12, student\_13, student\_14, student\_15, student\_16, n\_teacher, teacher\_1, teacher\_1.5, teacher\_2, teacher\_3, teacher\_4, teacher\_5, teacher\_6, teacher\_7, teacher\_8, teacher\_9, n\_date, date\_1, date\_2, date\_3, date\_4, date\_5, date\_6, date\_7, n\_city, city\_1, city\_2}\}$
- $\Sigma = \{\text{a, б, в, ..., я, ё}\} \cup \{\text{A, Б, В, ..., Я, Ё}\} \cup \{0, ..., 9\} \cup \{, , <, >, ., /, -, :, \backslash n\}$

- $s_{0_1} = \text{start}$
- $F = \{\text{end}\}$

На Рис. 1 представлен недетерминированный конечный автомат для проверки корректности титульного листа лабораторной работы или практики. В Табл. 1 представлены переходы полученного недетерминированного конечного автомата.

Таблица 1. Таблица переходов НДКА

| Текущее состояние   | Входной символ                                                 | Следующее состояние |
|---------------------|----------------------------------------------------------------|---------------------|
| <i>start</i>        | Министерство образования и науки Российской Федерации          | <i>minister</i>     |
| <i>minister</i>     | \n                                                             | <i>n_university</i> |
| <i>n_university</i> | Санкт-Петербургский политехнический университет Петра Великого | <i>university</i>   |
| <i>university</i>   | \n                                                             | <i>n_institut</i>   |
| <i>n_institut</i>   | Институт                                                       | <i>institut_1_1</i> |
| <i>n_institut</i>   | [А-ЯЁ]                                                         | <i>institut_2_1</i> |
| <i>institut_1_1</i> | \s                                                             | <i>institut_1_2</i> |
| <i>institut_1_2</i> | [а-яё]                                                         | <i>institut_1_3</i> |
| <i>institut_1_3</i> | [а-яё ]                                                        | <i>institut_1_3</i> |
| <i>institut_1_3</i> | \n                                                             | <i>n_school</i>     |
| <i>institut_2_1</i> | [а-яё]                                                         | <i>institut_2_1</i> |
| <i>institut_2_1</i> | \s                                                             | <i>institut_2_2</i> |
| <i>institut_2_2</i> | [а-яё]                                                         | <i>institut_2_1</i> |
| <i>institut_2_2</i> | институт                                                       | <i>institut_2_3</i> |
| <i>institut_2_3</i> | \n                                                             | <i>n_school</i>     |
| <i>n_school</i>     | Высшая школа                                                   | <i>school_1</i>     |
| <i>school_1</i>     | \s                                                             | <i>school_2</i>     |
| <i>school_2</i>     | [а-яё]                                                         | <i>school_3</i>     |
| <i>school_3</i>     | [а-яё ]                                                        | <i>school_3</i>     |
| <i>school_3</i>     | \n                                                             | <i>n_program</i>    |
| <i>n_program</i>    | Направление:                                                   | <i>program_1</i>    |
| <i>program_1</i>    | \s                                                             | <i>program_2</i>    |
| <i>program_2</i>    | \d{2}.\d{2}.\d{2}                                              | <i>program_3</i>    |
| <i>program_3</i>    | \s                                                             | <i>program_4</i>    |
| <i>program_4</i>    | [А-ЯЁ]                                                         | <i>program_5</i>    |
| <i>program_5</i>    | [а-яё ]                                                        | <i>program_5</i>    |
| <i>program_5</i>    | \n                                                             | <i>n_report</i>     |
| <i>n_report</i>     | Отчет о выполнении                                             | <i>report_1</i>     |
| <i>report_1</i>     | лабораторной работы №                                          | <i>report_2</i>     |
| <i>report_1</i>     | курсовой работы                                                | <i>report_4</i>     |
| <i>report_1</i>     | практики                                                       | <i>report_4</i>     |
| <i>report_2</i>     | [1-9]                                                          | <i>report_3</i>     |
| <i>report_3</i>     | [0-9]                                                          | <i>report_4</i>     |
| <i>report_3</i>     | \n                                                             | <i>n_name</i>       |
| <i>report_4</i>     | \n                                                             | <i>n_name</i>       |
| <i>n_name</i>       | <<                                                             | <i>name_1</i>       |
| <i>name_1</i>       | [А-ЯЁ]                                                         | <i>name_2</i>       |
| <i>name_2</i>       | [а-яё]                                                         | <i>name_2</i>       |
| <i>name_2</i>       | >>                                                             | <i>name_3</i>       |
| <i>name_3</i>       | \n                                                             | <i>n_variant</i>    |

| Текущее состояние  | Входной символ   | Следующее состояние |
|--------------------|------------------|---------------------|
| <i>n_variant</i>   | Вариант №        | <i>variant_1</i>    |
| <i>n_variant</i>   | Выполнил         | <i>student_1</i>    |
| <i>variant_1</i>   | [1-9]            | <i>variant_2</i>    |
| <i>variant_2</i>   | [0-9]            | <i>variant_3</i>    |
| <i>variant_2</i>   | \n               | <i>n_student</i>    |
| <i>variant_3</i>   | \n               | <i>n_student</i>    |
| <i>n_student</i>   | Выполнил         | <i>student_1</i>    |
| <i>student_1</i>   | a                | <i>student_1.5</i>  |
| <i>student_1</i>   | \s               | <i>student_2</i>    |
| <i>student_1.5</i> | \s               | <i>student_2</i>    |
| <i>student_2</i>   | студент          | <i>student_3</i>    |
| <i>student_3</i>   | ка               | <i>student_3.5</i>  |
| <i>student_3</i>   | \s               | <i>student_4</i>    |
| <i>student_3.5</i> | \s               | <i>student_4</i>    |
| <i>student_4</i>   | группы №         | <i>student_5</i>    |
| <i>student_5</i>   | \d{7}            | <i>student_6</i>    |
| <i>student_6</i>   | /                | <i>student_7</i>    |
| <i>student_7</i>   | \d{5}            | <i>student_8</i>    |
| <i>student_8</i>   | \s               | <i>student_9</i>    |
| <i>student_9</i>   | [А-ЯЁ]           | <i>student_10</i>   |
| <i>student_10</i>  | [а-яё]           | <i>student_10</i>   |
| <i>student_10</i>  | \s               | <i>student_11</i>   |
| <i>student_11</i>  | [А-ЯЁ]           | <i>student_12</i>   |
| <i>student_12</i>  | .                | <i>student_13</i>   |
| <i>student_13</i>  | \s               | <i>student_14</i>   |
| <i>student_14</i>  | [А-ЯЁ]           | <i>student_15</i>   |
| <i>student_15</i>  | .                | <i>student_16</i>   |
| <i>student_16</i>  | \n               | <i>n_teacher</i>    |
| <i>n_teacher</i>   | Проверил         | <i>teacher_1</i>    |
| <i>teacher_1</i>   | a                | <i>teacher_1.5</i>  |
| <i>teacher_1</i>   | \s               | <i>teacher_2</i>    |
| <i>teacher_1.5</i> | \s               | <i>teacher_2</i>    |
| <i>teacher_2</i>   | [А-ЯЁ]           | <i>teacher_3</i>    |
| <i>teacher_3</i>   | [а-яё]           | <i>teacher_3</i>    |
| <i>teacher_3</i>   | \s               | <i>teacher_4</i>    |
| <i>teacher_4</i>   | [А-ЯЁ]           | <i>teacher_5</i>    |
| <i>teacher_5</i>   | .                | <i>teacher_6</i>    |
| <i>teacher_6</i>   | \s               | <i>teacher_7</i>    |
| <i>teacher_7</i>   | [А-ЯЁ]           | <i>teacher_8</i>    |
| <i>teacher_8</i>   | .                | <i>teacher_9</i>    |
| <i>teacher_9</i>   | \n               | <i>n_date</i>       |
| <i>n_date</i>      | << _             | <i>date_1</i>       |
| <i>date_1</i>      | _                | <i>date_1</i>       |
| <i>date_1</i>      | >>               | <i>date_2</i>       |
| <i>date_2</i>      | \s               | <i>date_3</i>       |
| <i>date_3</i>      | _                | <i>date_4</i>       |
| <i>date_4</i>      | _                | <i>date_4</i>       |
| <i>date_4</i>      | \s               | <i>date_5</i>       |
| <i>date_5</i>      | \d{4}            | <i>date_6</i>       |
| <i>date_6</i>      | г.               | <i>date_7</i>       |
| <i>date_7</i>      | \n               | <i>n_city</i>       |
| <i>n_city</i>      | Санкт-Петербург, | <i>city_1</i>       |
| <i>city_1</i>      | \s               | <i>city_2</i>       |
| <i>city_2</i>      | \d{4}            | <i>end</i>          |



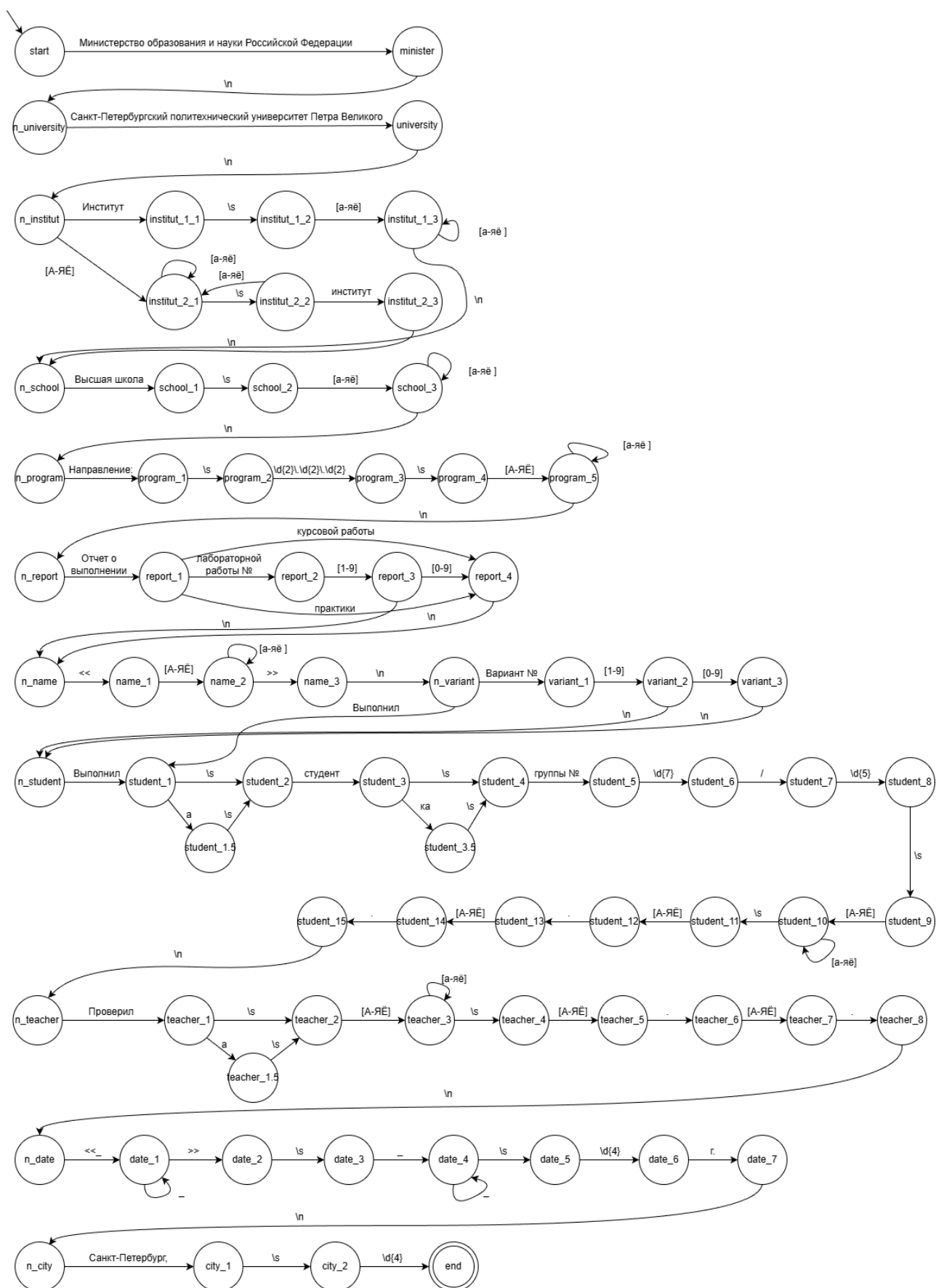


Рис. 1. Недетерминированный конечный автомат

### 1.3.2 Детерминированные конечные автоматы

Детерминированный конечный автомат (ДКА) определяется как:

$$(S_2, \Sigma, \sigma, s_{0_2}, F), \text{ где:}$$

$S_2$  – конечное множество состояний,  
 $\Sigma$  – конечный входной алфавит,  
 $\sigma : S_2 \times \Sigma \rightarrow S_2$  – функция переходов,  
 $s_{0_2} \in S_2$  – начальное состояние,  
 $F \subseteq S_2$  – множество допускающих состояний.

В данной работе:

- $S_2 = S_1 \cup \{fail\}$
- $\Sigma = \{a, б, в, ..., я, ё\} \cup \{A, Б, В, ..., Я, Ё\} \cup \{0, ..., 9\} \cup \{., <, >, ,, /, -, :, \backslash n\}$
- $s_{0_1} = start$
- $F = \{end\}$

На Рис. 2 представлен детерминированный конечный автомат для проверки корректности титульного листа лабораторной работы или практики. В Табл. 2 представлены переходы, добавленные в автомат для его детерминизации.

Таблица 2. Таблица переходов ДКА (дополнительные переходы)

| Текущее состояние   | Входной символ                                                                  | Следующее состояние |
|---------------------|---------------------------------------------------------------------------------|---------------------|
| <i>start</i>        | $\Sigma^* \backslash \{\text{Министерство...}\}$                                | <i>fail</i>         |
| <i>minister</i>     | $\Sigma^* \backslash \{\backslash n\}$                                          | <i>fail</i>         |
| <i>n_university</i> | $\Sigma^* \backslash \{\text{Санкт-Петербургский...}\}$                         | <i>fail</i>         |
| <i>university</i>   | $\Sigma^* \backslash \{\backslash n\}$                                          | <i>fail</i>         |
| <i>n_institut</i>   | $\Sigma^* \backslash \{\text{Институт, [А-ЯЁ]}\}$                               | <i>fail</i>         |
| <i>institut_1_1</i> | $\Sigma^* \backslash \{\backslash s\}$                                          | <i>fail</i>         |
| <i>institut_1_2</i> | $\Sigma^* \backslash \{\{a-яё\}, \text{институт}\}$                             | <i>fail</i>         |
| <i>institut_1_3</i> | $\Sigma^* \backslash \{\{a-яё\}, \varepsilon\}$                                 | <i>fail</i>         |
| <i>institut_2_1</i> | $\Sigma^* \backslash \{\{a-яё\}, \backslash s\}$                                | <i>fail</i>         |
| <i>institut</i>     | $\Sigma^* \backslash \{\backslash n\}$                                          | <i>fail</i>         |
| <i>n_school</i>     | $\Sigma^* \backslash \{\text{Высшая школа}\}$                                   | <i>fail</i>         |
| <i>school_1</i>     | $\Sigma^* \backslash \{\backslash s\}$                                          | <i>fail</i>         |
| <i>school_2</i>     | $\Sigma^* \backslash \{\{a-яё\}\}$                                              | <i>fail</i>         |
| <i>school_3</i>     | $\Sigma^* \backslash \{\{a-яё\}, \backslash n\}$                                | <i>fail</i>         |
| <i>n_program</i>    | $\Sigma^* \backslash \{\text{Направление:}\}$                                   | <i>fail</i>         |
| <i>program_1</i>    | $\Sigma^* \backslash \{\backslash s\}$                                          | <i>fail</i>         |
| <i>program_2</i>    | $\Sigma^* \backslash \{\backslash d\{2\}.\backslash d\{2\}.\backslash d\{2\}\}$ | <i>fail</i>         |
| <i>program_3</i>    | $\Sigma^* \backslash \{\backslash s\}$                                          | <i>fail</i>         |
| <i>program_4</i>    | $\Sigma^* \backslash \{\{A-ЯЁ\}\}$                                              | <i>fail</i>         |
| <i>program_5</i>    | $\Sigma^* \backslash \{\{a-яё\}, \backslash n\}$                                | <i>fail</i>         |

| Текущее состояние  | Входной символ                                                 | Следующее состояние |
|--------------------|----------------------------------------------------------------|---------------------|
| <i>n_report</i>    | $\Sigma^* \backslash \{\text{Отчет о выполнении}\}$            | <i>fail</i>         |
| <i>report_1</i>    | $\Sigma^* \backslash \{\text{лаборат..., курс..., практики}\}$ | <i>fail</i>         |
| <i>report_1_2</i>  | $\Sigma^* \backslash \{[1-9]\}$                                | <i>fail</i>         |
| <i>report_1_3</i>  | $\Sigma^* \backslash \{[0-9]\}$                                | <i>fail</i>         |
| <i>report_4</i>    | $\Sigma^* \backslash \{\backslash n\}$                         | <i>fail</i>         |
| <i>n_name</i>      | $\Sigma^* \backslash \{<<\}$                                   | <i>fail</i>         |
| <i>name_1</i>      | $\Sigma^* \backslash \{[A-Я\ddot{E}]\}$                        | <i>fail</i>         |
| <i>name_2</i>      | $\Sigma^* \backslash \{[a-я\ddot{e}], >>\}$                    | <i>fail</i>         |
| <i>name_3</i>      | $\Sigma^* \backslash \{\backslash n\}$                         | <i>fail</i>         |
| <i>n_variant</i>   | $\Sigma^* \backslash \{\text{Вариант №, Выполнил}\}$           | <i>fail</i>         |
| <i>variant_1</i>   | $\Sigma^* \backslash \{[1-9]\}$                                | <i>fail</i>         |
| <i>variant_2</i>   | $\Sigma^* \backslash \{[0-9]\}$                                | <i>fail</i>         |
| <i>variant_3</i>   | $\Sigma^* \backslash \{\backslash n\}$                         | <i>fail</i>         |
| <i>student_1</i>   | $\Sigma^* \backslash \{a, \backslash s\}$                      | <i>fail</i>         |
| <i>student_1_5</i> | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>student_2</i>   | $\Sigma^* \backslash \{\text{студент}\}$                       | <i>fail</i>         |
| <i>student_3</i>   | $\Sigma^* \backslash \{ка, \backslash s\}$                     | <i>fail</i>         |
| <i>student_3_5</i> | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>student_4</i>   | $\Sigma^* \backslash \{\text{группы №}\}$                      | <i>fail</i>         |
| <i>student_5</i>   | $\Sigma^* \backslash \{\backslash d\{7\}\}$                    | <i>fail</i>         |
| <i>student_6</i>   | $\Sigma^* \backslash \{/ \}$                                   | <i>fail</i>         |
| <i>student_7</i>   | $\Sigma^* \backslash \{\backslash d\{5\}\}$                    | <i>fail</i>         |
| <i>student_8</i>   | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>student_9</i>   | $\Sigma^* \backslash \{[A-Я\ddot{E}]\}$                        | <i>fail</i>         |
| <i>student_10</i>  | $\Sigma^* \backslash \{[a-я\ddot{e}], \backslash s\}$          | <i>fail</i>         |
| <i>student_11</i>  | $\Sigma^* \backslash \{[A-Я\ddot{E}]\}$                        | <i>fail</i>         |
| <i>student_12</i>  | $\Sigma^* \backslash \{.\}$                                    | <i>fail</i>         |
| <i>student_13</i>  | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>student_14</i>  | $\Sigma^* \backslash \{[A-Я\ddot{E}]\}$                        | <i>fail</i>         |
| <i>student_15</i>  | $\Sigma^* \backslash \{.\}$                                    | <i>fail</i>         |
| <i>student_16</i>  | $\Sigma^* \backslash \{\backslash n\}$                         | <i>fail</i>         |
| <i>n_teacher</i>   | $\Sigma^* \backslash \{\text{Проверил}\}$                      | <i>fail</i>         |
| <i>teacher_1</i>   | $\Sigma^* \backslash \{a, \backslash s\}$                      | <i>fail</i>         |
| <i>teacher_1_5</i> | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>teacher_2</i>   | $\Sigma^* \backslash \{[A-Я\ddot{E}]\}$                        | <i>fail</i>         |
| <i>teacher_3</i>   | $\Sigma^* \backslash \{[a-я\ddot{e}], \backslash s\}$          | <i>fail</i>         |
| <i>teacher_4</i>   | $\Sigma^* \backslash \{[A-Я\ddot{E}]\}$                        | <i>fail</i>         |
| <i>teacher_5</i>   | $\Sigma^* \backslash \{.\}$                                    | <i>fail</i>         |
| <i>teacher_6</i>   | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>teacher_7</i>   | $\Sigma^* \backslash \{[A-Я\ddot{E}]\}$                        | <i>fail</i>         |
| <i>teacher_8</i>   | $\Sigma^* \backslash \{.\}$                                    | <i>fail</i>         |
| <i>teacher_9</i>   | $\Sigma^* \backslash \{\backslash n\}$                         | <i>fail</i>         |
| <i>n_date</i>      | $\Sigma^* \backslash \{<<\}$                                   | <i>fail</i>         |
| <i>date_1</i>      | $\Sigma^* \backslash \{_\}$                                    | <i>fail</i>         |
| <i>date_2</i>      | $\Sigma^* \backslash \{_, >>\}$                                | <i>fail</i>         |
| <i>date_3</i>      | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>date_4</i>      | $\Sigma^* \backslash \{_\}$                                    | <i>fail</i>         |
| <i>date_5</i>      | $\Sigma^* \backslash \{_, \backslash s\}$                      | <i>fail</i>         |
| <i>date_6</i>      | $\Sigma^* \backslash \{\backslash d\{4\}\}$                    | <i>fail</i>         |
| <i>date_7</i>      | $\Sigma^* \backslash \{\Gamma\}$                               | <i>fail</i>         |
| <i>date_8</i>      | $\Sigma^* \backslash \{\backslash n\}$                         | <i>fail</i>         |
| <i>n_city</i>      | $\Sigma^* \backslash \{\text{Санкт-Петербург,}\}$              | <i>fail</i>         |
| <i>city_1</i>      | $\Sigma^* \backslash \{\backslash s\}$                         | <i>fail</i>         |
| <i>city_2</i>      | $\Sigma^* \backslash \{\backslash d\{4\}\}$                    | <i>fail</i>         |



## 2 Особенности реализации

### 2.1 Структуры данных

Класс TitlePageFSM – конечный автомат для валидации документа. Состоит из следующих полей:

- ALLOWED\_CHARS – множество допустимых символов алфавита;
- UPPER, LOWER, LOWER\_SPACE, DIGITS, NONZERO\_DIGITS – множества символов для проверки;
- states – множество состояний конечного автомата;
- text – проверяемая строка;
- pos – текущая позиция указателя в тексте;
- current\_state – текущее состояние автомата;
- transition\_table – таблица переходов.

Таблица transition\_table представляет собой словарь, в котором ключ – имя состояния, значение – список переходов из этого состояния. Каждый переход описывается словарём, в котором каждая запись состоит из условия перехода, имени следующего состояния, действия при переходе (например, сдвига указателя pos).

### 2.2 Инициализация автомата

При создании объекта класса инициализируются множество допустимых символов, множества символов для проверки, множество состояний автомата. Состояние автомата по умолчанию устанавливается на начальное "start". Вызывается метод \_set\_transitions для инициализации таблицы переходов.

**Листинг 1.** Метод \_\_init\_\_()

```
1 def __init__(self):
2     self.ALLOWED_CHARS = set(
3         'абвгдеёжзийклмнопрстуфхцчшщъыьэюя'
4         'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
5         '0123456789#<>_./-,: \n')
6     self.UPPER = set('АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ')
7     self.LOWER = set('абвгдеёжзийклмнопрстуфхцчшщъыьэюя')
8     self.LOWER_SPACE = self.LOWER | {' '}
9     self.DIGITS = set('0123456789')
10    self.NONZERO_DIGITS = set('123456789')
11
12    self.states = [
13        "start", "minister", "n_university", "university", "n_institut", "
14        institut_1_1", "institut_1_2",
15        "institut_1_3", "institut_2_1", "institut", "n_school", "school_1",
16        "school_2", "school_3",
17        "n_program", "program_1", "program_2", "program_3", "program_4", "
18        n_report", "report_1", "report_2",
```

```

16     "report_3", "report_4", "n_name", "name_1", "name_2", "name_3", "
17         n_variant", "variant_1", "variant_2",
18     "variant_3", "n_student", "student_1", "student_1_5", "student_2", "
19         student_3", "student_3_5",
20     "student_4", "student_5", "student_6", "student_7", "student_8", "
21         student_9", "student_10", "student_11",
22     "student_12", "student_13", "student_14", "student_15", "student_16"
23     , "n_teacher", "teacher_1",
24     "teacher_1_5", "teacher_2", "teacher_3", "teacher_4", "teacher_5", "
25         teacher_6", "teacher_7", "teacher_8",
26     "teacher_9", "n_date", "date_1", "date_2", "date_3", "date_4", "
27         date_5", "date_6", "date_7", "n_city",
28     "city_1", "city_2", "end"
29 ]
30
31 self.text = ""
32 self.pos = 0
33 self.current_state = "start"
34 self._set_transitions()

```

## set\_transitions

Инициализирует таблицу переходов конечного автомата.

Вход: объект класса TitlePageFSM.

Выход: инициализированная таблица переходов.

### Листинг 2. Метод set\_transitions()

```

1 def _set_transitions(self):
2     self.transition_table = {}
3
4     def add(s, cond, to, act=None):
5         self.transition_table.setdefault(s, []).append({
6             "c": cond,
7             "to": to,
8             "a": act
9         })
10
11     add("start",
12         lambda: self._check("Министерство образования и науки Российской Фед
13             ерации"),
14         "minister",
15         lambda: self._consume(len("Министерство образования и науки Российск
16             ой Федерации")))
17
18     add("minister",
19         lambda: self._check('\n'),
20         "n_university",
21         lambda: self._consume(1))
22
23     add("n_university",
24         lambda: self._check("Санкт-Петербургский политехнический университет
25             Петра Великого"),
26         "university",
27         lambda: self._consume(len("Санкт-Петербургский политехнический униве
28             рситет Петра Великого")))
29
30     add("university",
31         lambda: self._check('\n'),
32         "n_institut",
33         lambda: self._consume(1))

```

```

30
31 add("n_institut",
32     lambda: self._check("Институт"),
33     "institut_1_1",
34     lambda: self._consume(len("Институт")))
35
36 add("n_institut",
37     lambda: self._in(self._peek(), self.UPPER),
38     "institut_2_1",
39     lambda: self._consume(1))
40
41 add("institut_1_1",
42     lambda: self._check(' '),
43     "institut_1_2",
44     lambda: self._consume(1))
45
46 add("institut_1_2",
47     lambda: self._in(self._peek(), self.LOWER_SPACE),
48     "institut_1_3",
49     lambda: self._consume(1))
50
51 add("institut_1_3",
52     lambda: self._in(self._peek(), self.LOWER_SPACE),
53     "institut_1_3",
54     lambda: self._consume(1))
55
56 add("institut_1_3",
57     lambda: self._check('\n'),
58     "n_school",
59     lambda: self._consume(1))
60
61 add("institut_2_1",
62     lambda: self._in(self._peek(), self.LOWER_SPACE),
63     "institut_2_1",
64     lambda: self._consume(1))
65
66 add("institut_2_1",
67     lambda: self._check("институт"),
68     "institut",
69     lambda: self._consume(len("институт")))
70
71 add("institut",
72     lambda: self._check('\n'),
73     "n_school",
74     lambda: self._consume(1))
75
76 add("n_school",
77     lambda: self._check("Высшая школа"),
78     "school_1",
79     lambda: self._consume(len("Высшая школа")))
80
81 add("school_1",
82     lambda: self._check(' '),
83     "school_2",
84     lambda: self._consume(1))
85
86 add("school_2",
87     lambda: self._in(self._peek(), self.LOWER_SPACE),
88     "school_3",
89     lambda: self._consume(1))

```

```

90
91     add("school_3",
92         lambda: self._in(self._peek(), self.LOWER_SPACE),
93         "school_3",
94         lambda: self._consume(1))
95
96     add("school_3",
97         lambda: self._check('\n'),
98         "n_program",
99         lambda: self._consume(1))
100
101     add("n_program",
102         lambda: self._check("Направление: "),
103         "program_1",
104         lambda: self._consume(len("Направление: ")))
105
106     add("program_1",
107         lambda: self._check_code(),
108         "program_2",
109         lambda: self._consume(8))
110
111     add("program_2",
112         lambda: self._check(' '),
113         "program_3",
114         lambda: self._consume(1))
115
116     add("program_3",
117         lambda: self._in(self._peek(), self.UPPER),
118         "program_4",
119         lambda: self._consume(1))
120
121     add("program_4",
122         lambda: self._in(self._peek(), self.LOWER_SPACE),
123         "program_4",
124         lambda: self._consume(1))
125
126     add("program_4",
127         lambda: self._check('\n'),
128         "n_report",
129         lambda: self._consume(1))
130
131     add("n_report",
132         lambda: self._check("Отчет о выполнении "),
133         "report_1",
134         lambda: self._consume(len("Отчет о выполнении ")))
135
136     add("report_1",
137         lambda: self._check("лабораторной работы №"),
138         "report_2",
139         lambda: self._consume(len("лабораторной работы №")))
140
141     add("report_1",
142         lambda: self._check("курсовой работы"),
143         "report_4",
144         lambda: self._consume(len("курсовой работы")))
145
146     add("report_1",
147         lambda: self._check("практики"),
148         "report_4",
149         lambda: self._consume(len("практики")))

```



```

150
151 add("report_2",
152     lambda: self._in(self._peek(), self.NONZERO_DIGITS),
153     "report_3",
154     lambda: self._consume(1))
155
156 add("report_3",
157     lambda: self._check('\n'),
158     "n_name",
159     lambda: self._consume(1))
160
161 add("report_3",
162     lambda: self._in(self._peek(), self.DIGITS),
163     "report_4",
164     lambda: self._consume(1))
165
166 add("report_4",
167     lambda: self._check('\n'),
168     "n_name",
169     lambda: self._consume(1))
170
171 add("n_name",
172     lambda: self._check("<<"),
173     "name_1",
174     lambda: self._consume(len("<<")))
175
176 add("name_1",
177     lambda: self._in(self._peek(), self.UPPER),
178     "name_2",
179     lambda: self._consume(1))
180
181 add("name_2",
182     lambda: self._in(self._peek(), self.LOWER_SPACE),
183     "name_2",
184     lambda: self._consume(1))
185
186 add("name_2",
187     lambda: self._check(">>"),
188     "name_3",
189     lambda: self._consume(len(">>")))
190
191 add("name_3",
192     lambda: self._check('\n'),
193     "n_variant",
194     lambda: self._consume(1))
195
196 add("n_variant",
197     lambda: self._check("Вариант №"),
198     "variant_1",
199     lambda: self._consume(len("Вариант №")))
200
201 add("n_variant",
202     lambda: self._check("Выполнил"),
203     "student_1",
204     lambda: self._consume(len("Выполнил")))
205
206 add("variant_1",
207     lambda: self._in(self._peek(), self.NONZERO_DIGITS),
208     "variant_2",
209     lambda: self._consume(1))

```

```

210
211     add("variant_2",
212         lambda: self._in(self._peek(), self.DIGITS),
213         "variant_3",
214         lambda: self._consume(1))
215
216     add("variant_2",
217         lambda: self._check('\n'),
218         "n_student",
219         lambda: self._consume(1))
220
221     add("variant_3",
222         lambda: self._check('\n'),
223         "n_student",
224         lambda: self._consume(1))
225
226     add("n_student",
227         lambda: self._check("Выполнил"),
228         "student_1",
229         lambda: self._consume(len("Выполнил")))
230
231     add("student_1",
232         lambda: self._check("a"),
233         "student_1_5",
234         lambda: self._consume(1))
235
236     add("student_1",
237         lambda: self._check(' '),
238         "student_2",
239         lambda: self._consume(1))
240
241     add("student_1_5",
242         lambda: self._check(' '),
243         "student_2",
244         lambda: self._consume(1))
245
246     add("student_2",
247         lambda: self._check("студент"),
248         "student_3",
249         lambda: self._consume(len("студент")))
250
251     add("student_3",
252         lambda: self._check("ка"),
253         "student_3_5",
254         lambda: self._consume(len("ка")))
255
256     add("student_3",
257         lambda: self._check(' '),
258         "student_4",
259         lambda: self._consume(1))
260
261     add("student_3_5",
262         lambda: self._check(' '),
263         "student_4",
264         lambda: self._consume(1))
265
266     add("student_4",
267         lambda: self._check("группы №"),
268         "student_5",
269         lambda: self._consume(len("группы №")))

```

```

270
271     add("student_5",
272         lambda: self._check_n_digits(7),
273         "student_6",
274         lambda: self._consume(7))
275
276     add("student_6",
277         lambda: self._check('/'),
278         "student_7",
279         lambda: self._consume(1))
280
281     add("student_7",
282         lambda: self._check_n_digits(5),
283         "student_8",
284         lambda: self._consume(5))
285
286     add("student_8",
287         lambda: self._check(' '),
288         "student_9",
289         lambda: self._consume(1))
290
291     add("student_9",
292         lambda: self._in(self._peek(), self.UPPER),
293         "student_10",
294         lambda: self._consume(1))
295
296     add("student_10",
297         lambda: self._in(self._peek(), self.LOWER),
298         "student_10",
299         lambda: self._consume(1))
300
301     add("student_10",
302         lambda: self._check(' '),
303         "student_11",
304         lambda: self._consume(1))
305
306     add("student_11",
307         lambda: self._in(self._peek(), self.UPPER),
308         "student_12",
309         lambda: self._consume(1))
310
311     add("student_12",
312         lambda: self._check('.'),
313         "student_13",
314         lambda: self._consume(1))
315
316     add("student_13",
317         lambda: self._check(' '),
318         "student_14",
319         lambda: self._consume(1))
320
321     add("student_14",
322         lambda: self._in(self._peek(), self.UPPER),
323         "student_15",
324         lambda: self._consume(1))
325
326     add("student_15",
327         lambda: self._check('.'),
328         "student_16",
329         lambda: self._consume(1))

```

```

330
331     add("student_16",
332         lambda: self._check('\n'),
333         "n_teacher",
334         lambda: self._consume(1))
335
336     add("n_teacher",
337         lambda: self._check("Проверил"),
338         "teacher_1",
339         lambda: self._consume(len("Проверил")))
340
341     add("teacher_1",
342         lambda: self._check("a"),
343         "teacher_1_5",
344         lambda: self._consume(1))
345
346     add("teacher_1",
347         lambda: self._check(' '),
348         "teacher_2",
349         lambda: self._consume(1))
350
351     add("teacher_1_5",
352         lambda: self._check(' '),
353         "teacher_2",
354         lambda: self._consume(1))
355
356     add("teacher_2",
357         lambda: self._in(self._peek(), self.UPPER),
358         "teacher_3",
359         lambda: self._consume(1))
360
361     add("teacher_3",
362         lambda: self._in(self._peek(), self.LOWER),
363         "teacher_3",
364         lambda: self._consume(1))
365
366     add("teacher_3",
367         lambda: self._check(' '),
368         "teacher_4",
369         lambda: self._consume(1))
370
371     add("teacher_4",
372         lambda: self._in(self._peek(), self.UPPER),
373         "teacher_5",
374         lambda: self._consume(1))
375
376     add("teacher_5",
377         lambda: self._check('.'),
378         "teacher_6",
379         lambda: self._consume(1))
380
381     add("teacher_6",
382         lambda: self._check(' '),
383         "teacher_7",
384         lambda: self._consume(1))
385
386     add("teacher_7",
387         lambda: self._in(self._peek(), self.UPPER),
388         "teacher_8",
389         lambda: self._consume(1))

```

```

390
391     add("teacher_8",
392         lambda: self._check('.'),
393         "teacher_9",
394         lambda: self._consume(1))
395
396     add("teacher_9",
397         lambda: self._check('\n'),
398         "n_date",
399         lambda: self._consume(1))
400
401     add("n_date",
402         lambda: self._check("<<"),
403         "date_1",
404         lambda: self._consume(len("<<")))
405
406     add("date_1",
407         lambda: self._check('_'),
408         "date_1",
409         lambda: self._consume(1))
410
411     add("date_1",
412         lambda: self._check(">>"),
413         "date_2",
414         lambda: self._consume(len(">>")))
415
416     add("date_2",
417         lambda: self._check(' '),
418         "date_3",
419         lambda: self._consume(1))
420
421     add("date_3",
422         lambda: self._check('_'),
423         "date_4",
424         lambda: self._consume(1))
425
426     add("date_4",
427         lambda: self._check('_'),
428         "date_4",
429         lambda: self._consume(1))
430
431     add("date_4",
432         lambda: self._check(' '),
433         "date_5",
434         lambda: self._consume(1))
435
436     add("date_5",
437         lambda: self._check_n_digits(4),
438         "date_6",
439         lambda: self._consume(4))
440
441     add("date_6",
442         lambda: self._check("r."),
443         "date_7",
444         lambda: self._consume(len("r.")))
445
446     add("date_7",
447         lambda: self._check('\n'),
448         "n_city",
449         lambda: self._consume(1))

```

```

450
451     add("n_city",
452         lambda: self._check("Санкт-Петербург,"),
453         "city_1",
454         lambda: self._consume(len("Санкт-Петербург,")))
455
456     add("city_1",
457         lambda: self._check(' '),
458         "city_2",
459         lambda: self._consume(1))
460
461     add("city_2",
462         lambda: self._check_n_digits(4) and self.pos + 4 == len(self.text),
463         "end",
464         lambda: self._consume(4))

```

## 2.3 Функции

### 2.3.1 Вспомогательные функции

#### peek()

Возвращает текущий символ текста по позиции pos без перемещения указателя. Если позиция за границами текста, возвращает None.

Вход: объект класса TitlePageFSM.

Выход: символ или None.

#### Листинг 3. Метод peek()

```

1 def _peek(self):
2     return self.text[self.pos] if self.pos < len(self.text) else None

```

#### consume()

Сдвигает позицию автомата (pos) относительно строки вперёд на заданное количество символов.

Вход: объект класса TitlePageFSM, количество символов.

Выход: объект класса TitlePageFSM со сдвинутой позицией автомата.

#### Листинг 4. Метод consume()

```

1 def consume(self, n=1):
2     self.pos += n

```

#### check\_text()

Нормализует текст, проверяет недопустимые символы, затем вызывает основной метод проверки корректности строки и возвращает результат этой проверки.

Вход: объект класса TitlePageFSM, text – проверяемая строка.

Выход: результат проверки корректности строки, список недопустимых символов.

#### Листинг 5. Функция check\_text()

```

1 def check_text(text, fsm):
2     text = text.replace('\r\n', '\n').replace('\r', '\n').rstrip('\n')

```

```

3
4     invalid_chars = set(text) - fsm.ALLOWED_CHARS
5     if invalid_chars:
6         return 'invalid_chars', invalid_chars
7
8     if fsm.validate(text):
9         return 'valid', None
10    else:
11        return 'invalid', None

```

### 2.3.2 Функции, проверяющие совпадение символов

#### in()

Проверяет принадлежность символа к классу заглавных кириллических букв.

Вход: символ, множество символов.

Выход: True/False.

#### Листинг 6. Метод in()

```

1 def _in(self, c, charset):
2     return c is not None and c in charset

```

#### check()

Проверяет, совпадает ли текст с позиции pos с ожидаемой строкой.

Вход: объект класса TitlePageFSM, ожидаемая строка.

Выход: True/False.

#### Листинг 7. Метод check()

```

1 def _check(self, s):
2     return self.text.startswith(s, self.pos)

```

#### check\_n\_digits()

Проверяет последовательность символов на совпадение последовательностью цифр.

Вход: объект класса TitlePageFSM.

Выход: True/False.

#### Листинг 8. Метод check\_char()

```

1 def _check_char(self, ch):
2     return self._peek() == ch

```

### 2.3.3 Функция, реализующая логику конечного автомата

#### validate()

Основная функция проверки строки. Инициализирует начальное состояние и позицию в тексте, затем в цикле для текущего состояния перебирает все возможные переходы и выбирает первый, условие которого выполняется для текущего фрагмента строки; при этом, если у перехода есть действие, оно сдвигает позицию, после чего автомат переходит в следующее состояние и продолжает работу. Если ни один переход не подходит, функция возвращает

False, что означает, что строка не соответствует шаблону. Если же автомат успешно доходит до конечного состояния end, это означает, что вся строка корректно распознана, и функция возвращает True. Вход: объект класса TitlePageFSM, text – проверяемая строка.

Выход: True/False.

#### Листинг 9. Метод validate()

```
1 def validate(self, text):
2     self.text = text
3     self.pos = 0
4     self.current_state = "start"
5
6     while self.current_state != "end":
7         transitions = self.transition_table.get(self.current_state, [])
8         matched = False
9
10        for tr in transitions:
11            if tr["c"]():
12                if tr["a"]:
13                    tr["a]()
14                    self.current_state = tr["to"]
15                    matched = True
16                    break
17
18        if not matched:
19            return False
20
21    return True
```

### 2.3.4 Функции для генерации строк

#### random\_cyrillic()

Генерирует случайную строку из строчных кириллических букв заданной длины.

Вход: минимальная и максимальная длины.

Выход: строка.

#### Листинг 10. Функция random\_cyrillic()

```
1 def random_cyrillic(min_len, max_len):
2     length = random.randint(min_len, max_len)
3     chars = 'абвгдеёжзийклмнопрстуфхцчшщъыьэюя'
4     return ''.join(random.choice(chars) for _ in range(length))
```

#### random\_cyrillic\_capital()

Генерирует случайную заглавную кириллическую букву.

Вход: –

Выход: символ.

#### Листинг 11. Функция random\_cyrillic\_capital()

```
1 def random_cyrillic_capital():
2     chars = 'АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯ'
3     return random.choice(chars)
```



## generate\_random\_string()

Последовательно генерирует каждую часть документа в соответствии с конечным автоматом. Для каждого элемента, имеющего варианты в шаблоне, случайным образом выбирается один из допустимых вариантов. Последовательности символов генерируются случайным образом из допустимого множества символов.

Вход: –

Выход: строка, соответствующая регулярному выражению.

### Листинг 12. Функция generate\_random\_string()

```
1 def generate_random_string():
2     # институт
3     if random.choice([True, False]):
4         institute = f"{random_cyrillic_capital()}{random_cyrillic(1, 15)} ин
5             ститут"
6     else:
7         institute = f"Институт {random_cyrillic(1, 15)}"
8
9     # направление
10    code = f"{random.randint(10, 99)}.{random.randint(10, 99)}.{random.
11        randint(10, 99)}"
12    direction = f"Направление: {code} {random_cyrillic_capital()}{
13        random_cyrillic(1, 15)}"
14
15    # тип работы
16    report_types = [
17        f"лабораторной работы №{random.randint(1, 9)}{random.randint(0, 9)}"
18        ,
19        "курсовой работы",
20        "практики"
21    ]
22    report_type = random.choice(report_types)
23
24    # тема
25    topic = "<<" + random_cyrillic_capital() + random_cyrillic(15, 50) + ">>"
26
27    # вариант
28    variant = ""
29    if random.choice([True, False]):
30        variant = f"Вариант №{random.randint(1, 9)}{random.randint(0, 9)}\n"
31
32    # студент
33    gender = random.choice(['м', 'ж'])
34    student_verb = "Выполнил" if gender == 'м' else "Выполнила"
35    student_noun = "студент" if gender == 'м' else "студентка"
36    student_line = f"{student_verb} {student_noun} группы №{random.randint
37        (1000000, 9999999)}/{random.randint(10000, 99999)} {
38        random_cyrillic_capital()}{random_cyrillic(2, 15)} {
39        random_cyrillic_capital()}. {random_cyrillic_capital()}."
40
41    # преподаватель
42    teacher_gender = random.choice(['м', 'ж'])
43    teacher_verb = "Проверил" if teacher_gender == 'м' else "Проверила"
44    teacher_line = f"{teacher_verb} {random_cyrillic_capital()}{
45        random_cyrillic(2, 15)} {random_cyrillic_capital()}. {
46        random_cyrillic_capital()}."
```

```

38
39 # дата
40 year = random.randint(2000, 2026)
41 signature = f"<<{'_' * random.randint(5, 15)}>> {'_' * random.randint(5,
42     15)} {year}г."
43
44 city = f"Санкт-Петербург, {year}"
45
46 lines = [
47     "Министерство образования и науки Российской Федерации",
48     "Санкт-Петербургский политехнический университет Петра Великого",
49     institute,
50     f"Высшая школа {random_cyrillic(5, 15)}",
51     direction,
52     f"Отчет о выполнении {report_type}",
53     topic,
54     variant + student_line if variant else student_line,
55     teacher_line,
56     signature,
57     city
58 ]
59
60 return '\n'.join(lines)

```

### 2.3.5 Меню

**main()** Отображает меню, обрабатывает выбор пользователя, вызывает соответствующие функции проверки или генерации, выводит результат.

#### Листинг 13. Функция main()

```

1 def main():
2     fsm = TitlePageFSM()
3
4     while True:
5         print("\n" + "-" * 60)
6         print("Меню:")
7         print("1. Ввести текст вручную")
8         print("2. Загрузить из файла")
9         print("3. Сгенерировать случайный верный документ")
10        print("4. Выход")
11
12        choice = input("\nВаш выбор (1-4): ").strip()
13
14        if choice == '1':
15            print("\nВведите текст (пустая строка для завершения):")
16            lines = []
17            while True:
18                line = input()
19                if line == '' and lines:
20                    break
21                lines.append(line)
22            text = '\n'.join(lines)
23
24        elif choice == '2':
25            filename = input("Введите имя файла: ").strip()
26            text = load_from_file(filename)
27

```

```

28     elif choice == '3':
29         text = generate_random_string()
30         print("\n" + "=" * 60)
31         print("СГЕНЕРИРОВАННЫЙ ДОКУМЕНТ:")
32         print("=" * 60)
33         print(text)
34
35     elif choice == '4':
36         print("Выход.")
37         break
38
39     else:
40         print("Неверный выбор!")
41         continue
42
43     if choice == '1' or choice == '2':
44         print("\n" + "=" * 60)
45         print("РЕЗУЛЬТАТ ПРОВЕРКИ")
46         print("=" * 60)
47
48         result, data = check_text(text, fsm)
49
50         if result == 'valid':
51             print("Строка СООТВЕТСТВУЕТ шаблону")
52         elif result == 'invalid':
53             print("Строка НЕ СООТВЕТСТВУЕТ шаблону")
54         else:
55             print("СИМВОЛЫ НЕ ИЗ АЛФАВИТА:")
56             invalid_list = sorted(list(data))
57             display_chars = []
58             for ch in invalid_list:
59                 if ch == '\n':
60                     display_chars.append('\\n')
61                 elif ch == '\r':
62                     display_chars.append('\\r')
63                 elif ch == '\t':
64                     display_chars.append('\\t')
65                 elif ch == ' ':
66                     display_chars.append('пробел')
67                 else:
68                     display_chars.append(ch)
69             print(f"Символы: {display_chars}")

```

### 3 Результаты работы программы

Далее представлены результаты работы программы для разных входных строк.

#### Пример 1. Корректная строка

```
1 Министерство образования и науки Российской Федерации
2 Санкт-Петербургский политехнический университет Петра Великого
3 Институт азбххтфылнёххш
4 Высшая школа свеяки
5 Направление: 76.73.54 Сеегур
6 Отчет о выполнении практики
7 <<Вэюхьшрюяцузшгёфьшмуьхуу>>
8 Вариант №62
9 Выполнила студентка группы №7839205/43083 Всфцсуерйъфщв Ъ. Ъ.
10 Проверила Дншгоуьуаэ Ц. К.
11 <<_____>> _____ 2008г.
12 Санкт-Петербург, 2008
13
14 =====
15 РЕЗУЛЬТАТ ПРОВЕРКИ
16 =====
17 Строка СООТВЕТСТВУЕТ шаблону
```

#### Пример 2. Корректная строка

```
1 Санкт-Петербургский политехнический университет Петра Великого
2 Институт компьютерных наук и кибербезопасности
3 Высшая школа технологий искусственного интеллекта
4 Направление: 02.03.01 Математика и компьютерные науки
5 Отчет о выполнении лабораторной работы №1
6 <<Лексический анализатор>>
7 Вариант №8
8 Выполнила студентка группы №5130201/30101 Лаишевкина А. М.
9 Проверил Востров А. В.
10 <<_____>> _____ 2026г.
11 Санкт-Петербург, 2026
12
13 =====
14 РЕЗУЛЬТАТ ПРОВЕРКИ
15 =====
16 Строка СООТВЕТСТВУЕТ шаблону
```

#### Пример 3. Корректная строка

```
1 Министерство образования и науки Российской Федерации
2 Санкт-Петербургский политехнический университет Петра Великого
3 Институт зцххгз
4 Высшая школа аряппдокезгъжё
5 Направление: 94.28.41 Мвэдьоёзцзцйт
6 Отчет о выполнении практики
7 <<Яхэоцярзэакмгцкъвхгё>>
8 Вариант №15
9 Выполнил студент группы №2247546/50662 Зоёкшзёгчддъэкк Л. Щ.
10 Проверила Асжцтусы Й. Ф.
11 <<_____>> _____ 2015г.
12 Санкт-Петербург, 2015
13
14 =====
15 РЕЗУЛЬТАТ ПРОВЕРКИ
16 =====
```

17 Строка СООТВЕТСТВУЕТ шаблону

#### Пример 4. Некорректная строка

```
1 Министерство образования и науки Российской Федерации
2 Санкт-Петербургский политехнический университет Петра Великого
3 Институт ьтчзсзщнуьъф
4 Высшая школа ачщезхчэнэпчид
5 Отчет о выполнении курсовой работы
6 <<Ыфрсдоавцблүйкнхъфэудрбфщющнацжсяю>>
7 Выполнила студентка группы №7396706/41485 Шокцъшяолш С. Б.
8 Проверил Жввфусцлжг Щ. С.
9 <<_____>> _____ 2012г.
10 Санкт-Петербург, 2012
11
12 =====
13 РЕЗУЛЬТАТ ПРОВЕРКИ
14 =====
15 Строка НЕ СООТВЕТСТВУЕТ шаблону
```

#### Пример 5. Недопустимые символы

```
1 Министерство образования и науки Российской Федерации
2 Санкт-Петербургский политехнический университет Петра Великого
3 #&%
4
5 =====
6 РЕЗУЛЬТАТ ПРОВЕРКИ
7 =====
8 СИМВОЛЫ НЕ ИЗ АЛФАВИТА:
9 Символы: ['#', '%', '&', 't', 'u']
```

#### Пример 6. Пустая строка

```
1 =====
2 РЕЗУЛЬТАТ ПРОВЕРКИ
3 =====
4 Строка НЕ СООТВЕТСТВУЕТ шаблону
```

Далее представлен пример сгенерированной строки.

```
1 Министерство образования и науки Российской Федерации
2 Санкт-Петербургский политехнический университет Петра Великого
3 Институт ррслёчн
4 Высшая школа еефнимъч
5 Направление: 91.65.68 Ёрлёгэтоэъё
6 Отчет о выполнении курсовой работы
7 <<Ёлысдхэсхэяфитмзкьпъкдфхпомяъ>>
8 Вариант №11
9 Выполнила студентка группы №4526638/69012 Экжнщхчвуыюз М. Ш.
10 Проверила Норзмхсчсц Д. В.
11 <<_____>> _____ 2022г.
12 Санкт-Петербург, 2022
```

## Заключение

В ходе данной работы было построено регулярное выражение, соответствующее шаблону титульного листа лабораторной работы или практики, недетерминированный конечный автомат, реализующий распознавание этого регулярного выражения, детерминированный конечный автомат. Описываемый язык относится к классу регулярных языков, поскольку задаётся регулярным выражением и распознаётся конечным автоматом, что согласуется с теоремой Клини о равнозначности регулярных выражений и конечных автоматов. Также была реализована программа на Python, которая проверяет введенный текст на соответствие регулярному выражению и генерирует верные строки.

### **Достоинства:**

1. Программа генерирует реалистичные даты.
2. Автомат распознает форму женского рода «студентка».
3. В программе есть возможность загрузить текст из файла.

### **Недостатки:**

1. Автомат не предполагает отсутствие названия работы.

### **Масштабирование:**

1. Добавление формы женского рода «преподавательница».
2. Добавление новых строк на титульный лист.

## Список литературы

1. Секция «Телематика» – <https://tema.spbstu.ru/compiler/> (дата обращения – 04.03.2026)